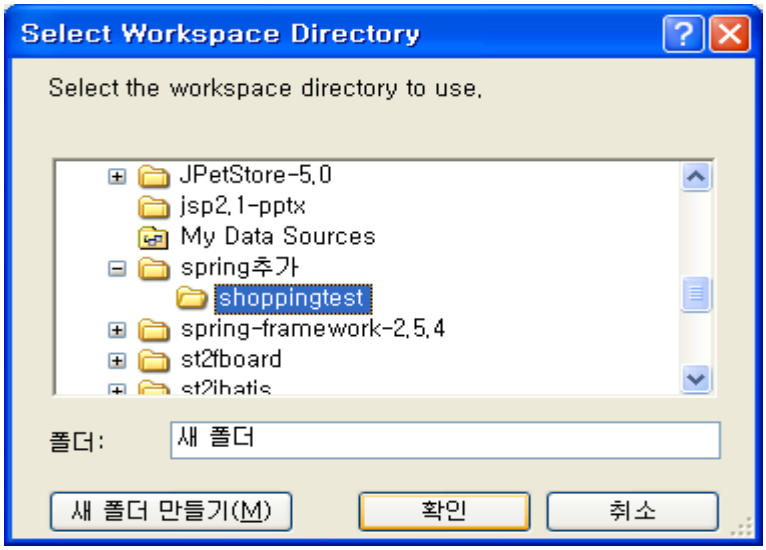
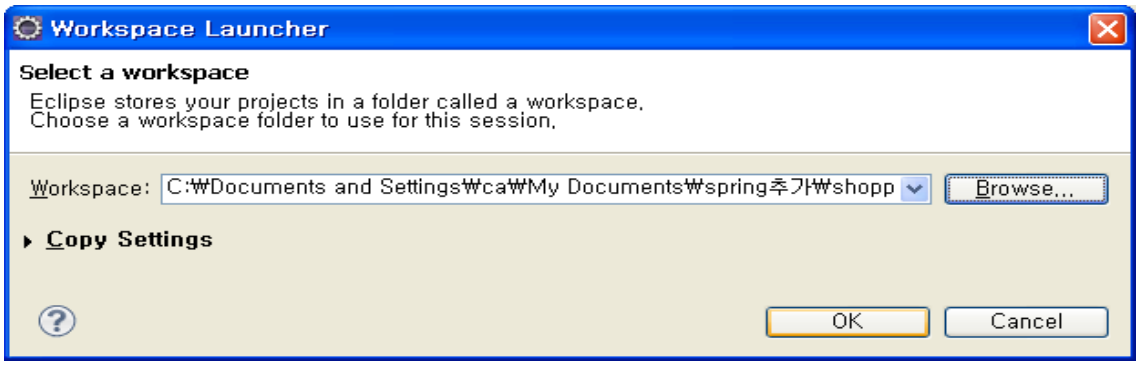
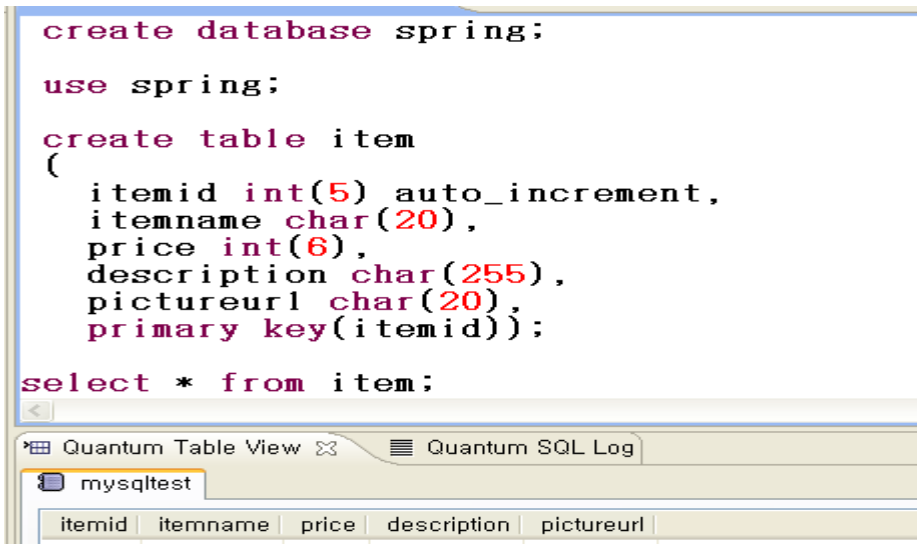
Spring으로 쇼핑몰을 구축하기위한 shoppingtest라는 작업폴더를 이클립스에서 설정





1.spring이라는 데이터베이스에 item이라는 상품테이블을 작성하고 데이터를 입력함



```
create database spring;

use spring;

create table item
(
   itemid int(5) auto_increment,
   itemname char(20),
   price int(6),
   description char(255),
   pictureurl char(20),
   primary key(itemid));

select * from item;
```

Quantum Table View | Quantum SQL Log

mysqltest

itemid	itemname	price	description	pictureurl

```
insert into item values(1,'레몬',300,'레몬에 포함되어있는 쿠엔산은 피로회복에 좋습
insert into item values(2,'오렌지',2000,'비타민C가 풍부합니다. 플래쉬 쥬스도 추천
insert into item values(3,'키위',300,'비타민C가 매우 풍부합니다.다이어트와 미용에
insert into item values(4,'파란사과',500,'단맛이 강한 향기가 상큼한 파란사과입니다
insert into item values(5,'블루베리',500,'블루베리에 포함된 안토시아닌은 눈피로에
insert into item values(6,'체리',1000,'체리는 맛이 단 성분이 많고 피로회복에 잘 듣
insert into item values(7,'메론',1000,'메론에는 비타민A와 칼륨이 많이 포함되어있습
insert into item values(8,'수박',2000,'수분이 풍부한 작인 수박입니다','suika.jpg'
insert into item values(9,'파인애플',2000,'비타민 B1이 비타민 B2가 풍부합니다.요리
```

Quantum Table View | Quantum SQL Log

mysqltest

itemid	itemname	price	description	pictureurl
1	레몬	300	레몬에 포함되어있는 쿠엔산은 피로회복에 좋습니다.비타민C도 풍부합니다	lemon.jpg
2	오렌지	2000	비타민C가 풍부합니다. 플래쉬 쥬스도 추천합니다	orange.jpg
3	키위	300	비타민C가 매우 풍부합니다.다이어트와 미용에도 매우 좋습니다.	kiui.jpg
4	파란사과	500	단맛이 강한 향기가 상큼한 파란사과입니다.	aoringo.jpg
5	블루베리	500	블루베리에 포함된 안토시아닌은 눈피로에 좋습니다.	blueberry.jpg
6	체리	1000	체리는 맛이 단 성분이 많고 피로회복에 잘 듣습니다.	sakuranbo.jpg
7	메론	1000	메론에는 비타민A와 칼륨이 많이 포함되어있습니다.	melon.jpg
8	수박	2000	수분이 풍부한 작인 수박입니다	suika.jpg
9	파인애플	2000	비타민 B1이 비타민 B2가 풍부합니다.요리에도 사용할 수 있습니다	pine.jpg

동적 프로젝트를 구성할 때 컴파일시킬 클래스의 위치만 변경시킨다.

Java Build Path

Source | Projects | Libraries | Order and Export

Source folders on build path:

shopping-1/src

Add Folder...

Link Source...

Edit...

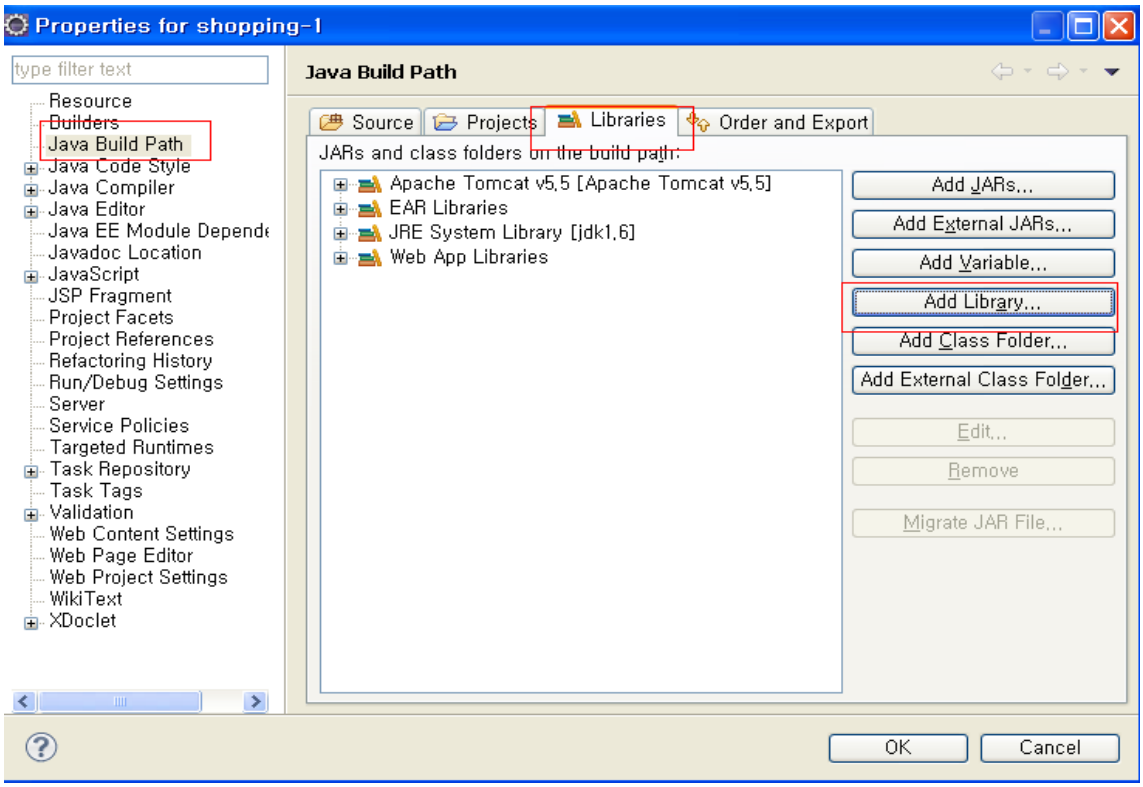
Remove

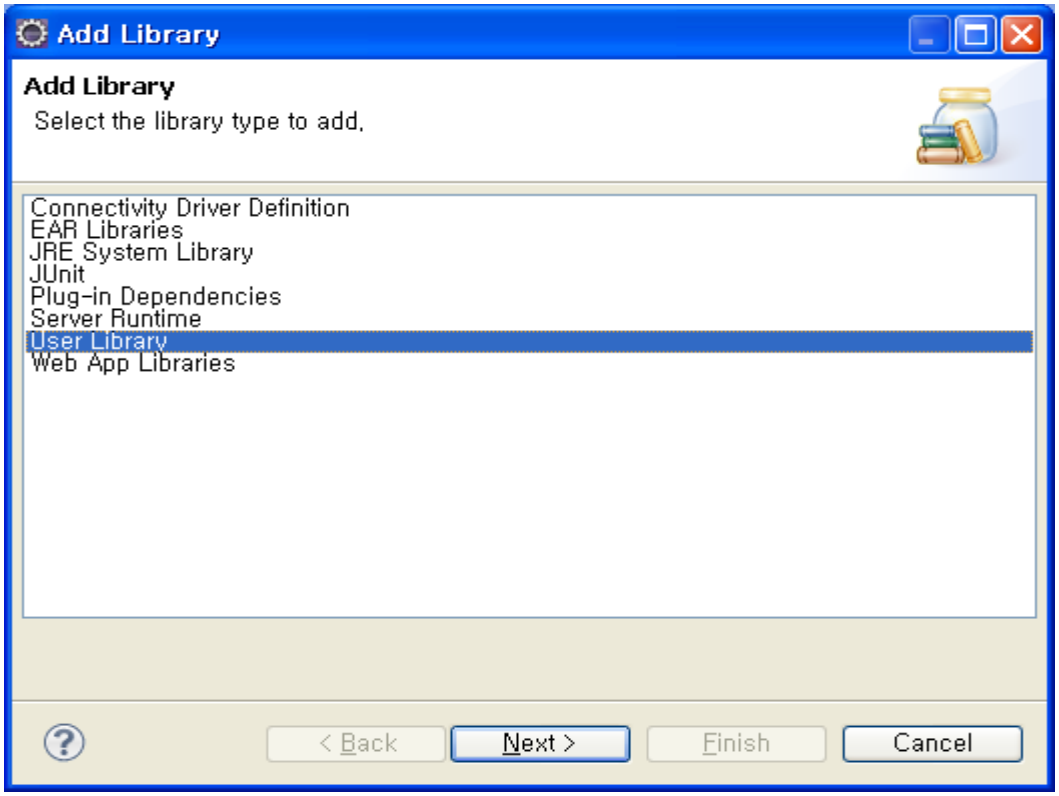
Allow output folders for source folders

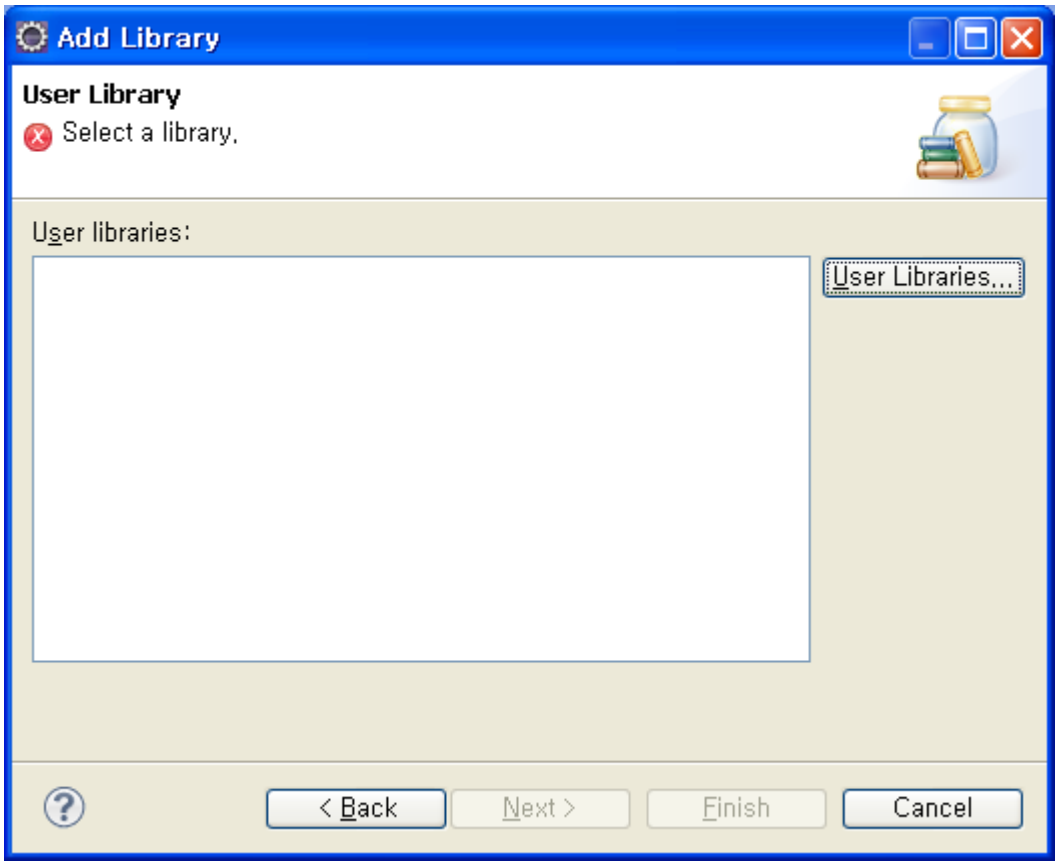
Default output folder:

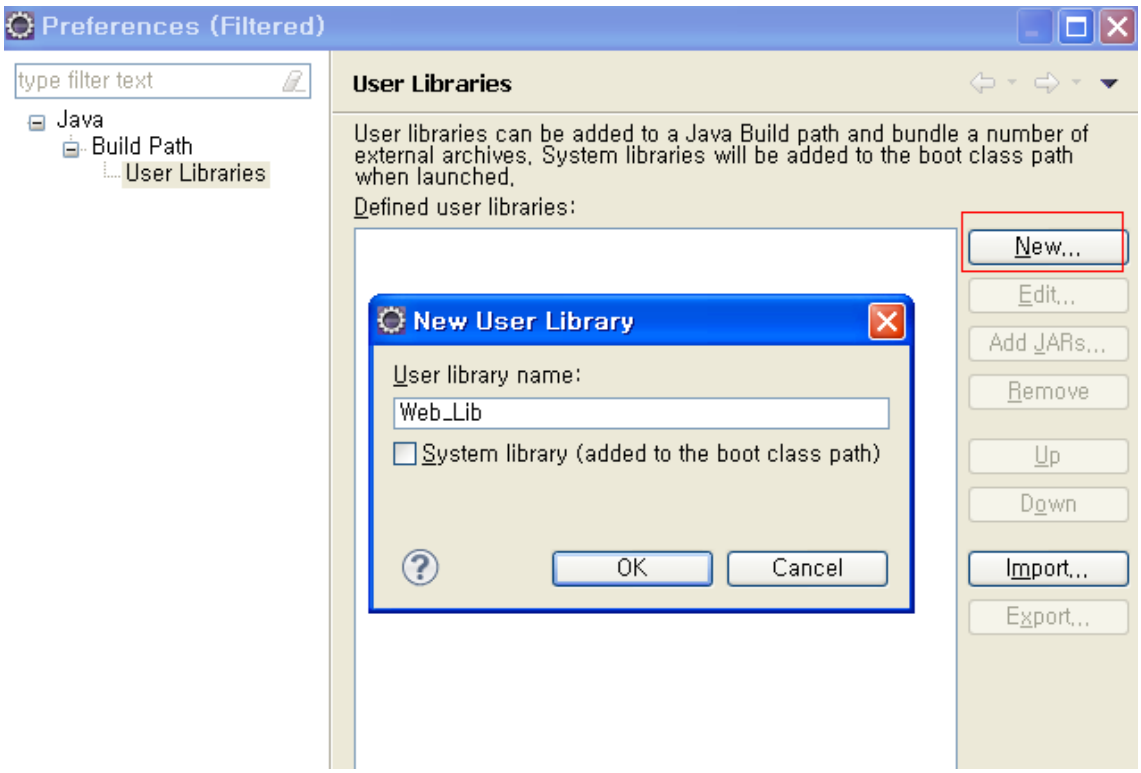
shopping-1/WebContent/WEB-INF/classes

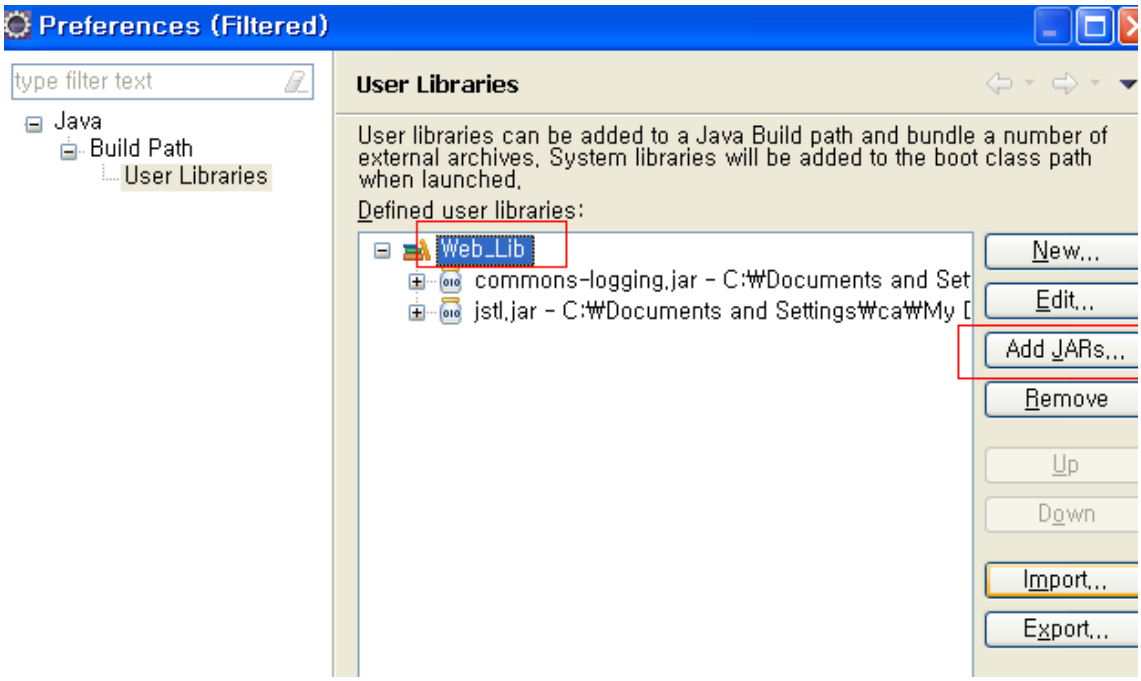
Browse...

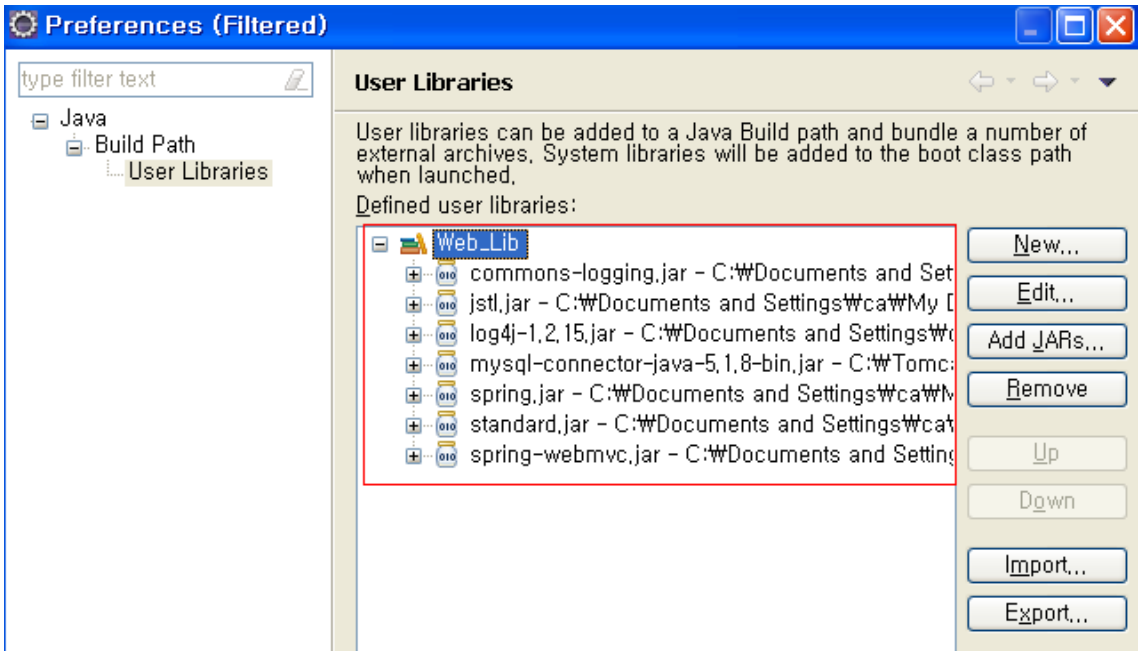

Properties for shopping-1
type filter text
Resource
Builders
Java Build Path
Java Code Style
Java Compiler
Java Editor
Java EE Module Depende
Javadoc Location
JavaScript
JSP Fragment
Project Facets
Project References
Refactoring History
Run/Debug Settings
Server
Service Policies
Targeted Runtimes
Task Repository
Task Tags
Validation
Web Content Settings
Web Page Editor
Web Project Settings
WikiText
XDoclet
Java Build Path
Source
Projects
Libraries
Order and Export
JARs and class folders on the build path:
Apache Tomcat v5.5 [Apache Tomcat v5.5]
EAR Libraries
JRE System Library [jdk1.6]
Web App Libraries
Add JARs...
Add External JARs...
Add Variable...
Add Library...
Add Class Folder...
Add External Class Folder...
Edit...
Remove
Migrate JAR File...
OK
Cancel

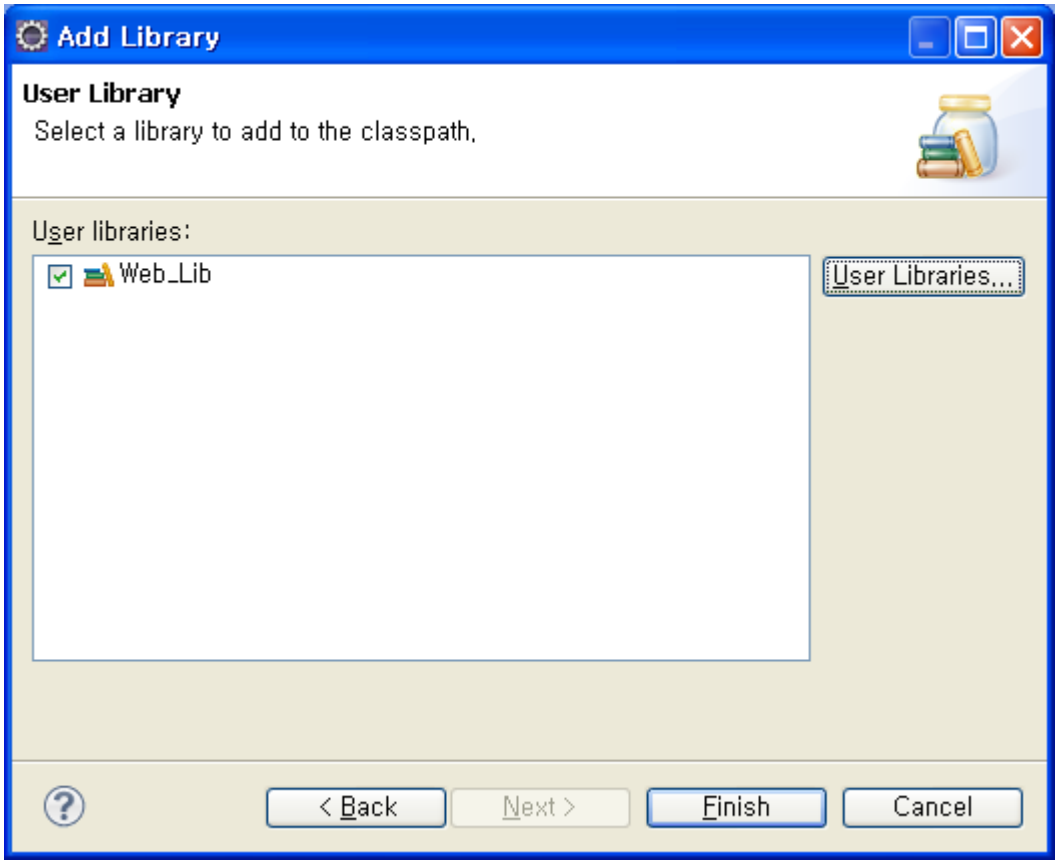

Add Library
Add Library
Select the library type to add.
Connectivity Driver Definition
EAR Libraries
JRE System Library
JUnit
Plug-in Dependencies
Server Runtime
User Library
Web App Libraries
< Back
Next >
Finish
Cancel

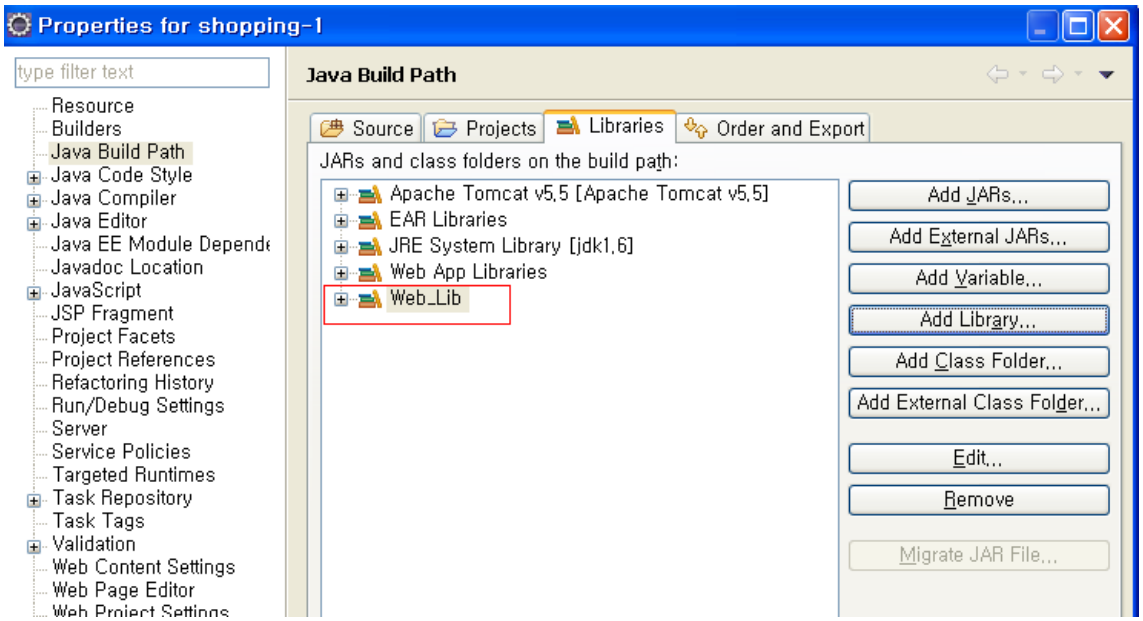

Add Library
User Library
Select a library.
User libraries:
User Libraries...
< Back
Next >
Finish
Cancel


Preferences (Filtered)
type filter text
Java
Build Path
User Libraries
User Libraries
User libraries can be added to a Java Build path and bundle a number of external archives. System libraries will be added to the boot class path when launched.
Defined user libraries:
New...
Edit...
Add JARs...
Remove
Up
Down
Import...
Export...
New User Library
User library name:
Web_Lib
System library (added to the boot class path)
OK
Cancel

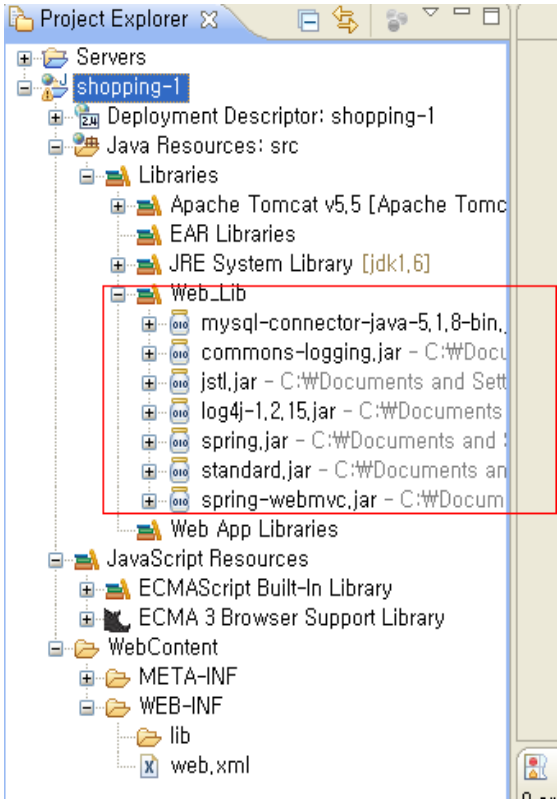

Preferences (Filtered)
type filter text
Java
Build Path
User Libraries
User Libraries
User libraries can be added to a Java Build path and bundle a number of external archives. System libraries will be added to the boot class path when launched.
Defined user libraries:
Web_Lib
commons-logging.jar - C:₩Documents and Set
jstl.jar - C:₩Documents and Settings₩ca₩My [
New...
Edit...
Add JARs...
Remove
Up
Down
Import...
Export...

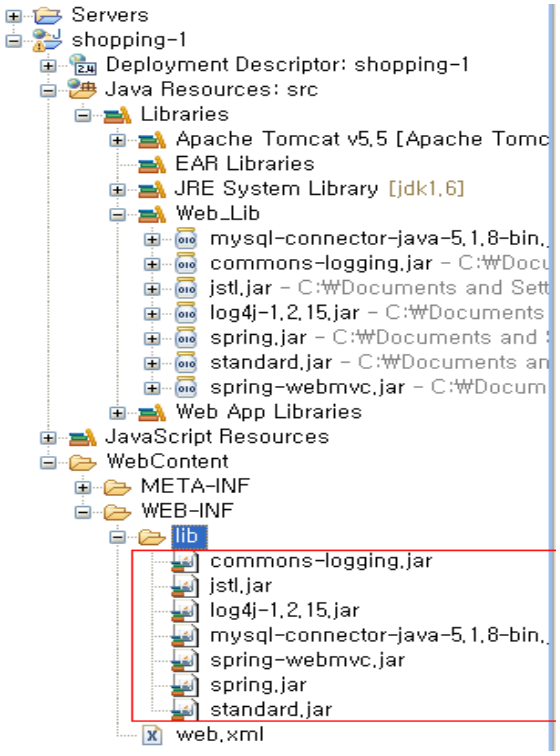

Preferences (Filtered)
type filter text
Java
Build Path
User Libraries
User Libraries
User libraries can be added to a Java Build path and bundle a number of external archives. System libraries will be added to the boot class path when launched.
Defined user libraries:
Web_Lib
commons-logging.jar - C:₩Documents and Set
jstl.jar - C:₩Documents and Settings₩ca₩My [
log4j-1.2.15.jar - C:₩Documents and Settings₩
mysql-connector-java-5.1.8-bin.jar - C:₩Tomc
spring.jar - C:₩Documents and Settings₩ca₩
standard.jar - C:₩Documents and Settings₩ca
spring-webmvc.jar - C:₩Documents and Setting
New...
Edit...
Add JARs...
Remove
Up
Down
Import...
Export...


Add Library
User Library
Select a library to add to the classpath.
User libraries:
Web_Lib
User Libraries...
< Back
Next >
Finish
Cancel


Properties for shopping-1
type filter text
Resource
Builders
Java Build Path
Java Code Style
Java Compiler
Java Editor
Java EE Module Depend
Javadoc Location
JavaScript
JSP Fragment
Project Facets
Project References
Refactoring History
Run/Debug Settings
Server
Service Policies
Targeted Runtimes
Task Repository
Task Tags
Validation
Web Content Settings
Web Page Editor
Web Project Settings
Java Build Path
Source
Projects
Libraries
Order and Export
JARs and class folders on the build path:
Apache Tomcat v5.5 [Apache Tomcat v5.5]
EAR Libraries
JRE System Library [jdk1.6]
Web App Libraries
Web_Lib
Add JARs...
Add External JARs...
Add Variable...
Add Library...
Add Class Folder...
Add External Class Folder...
Edit...
Remove
Migrate JAR File...

최종 추가된 사용자 정의 라이브러리 파일





설정 파일(Web.xml)

```
xmlns:web= http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
id="WebApp_ID" version="2.5">
```

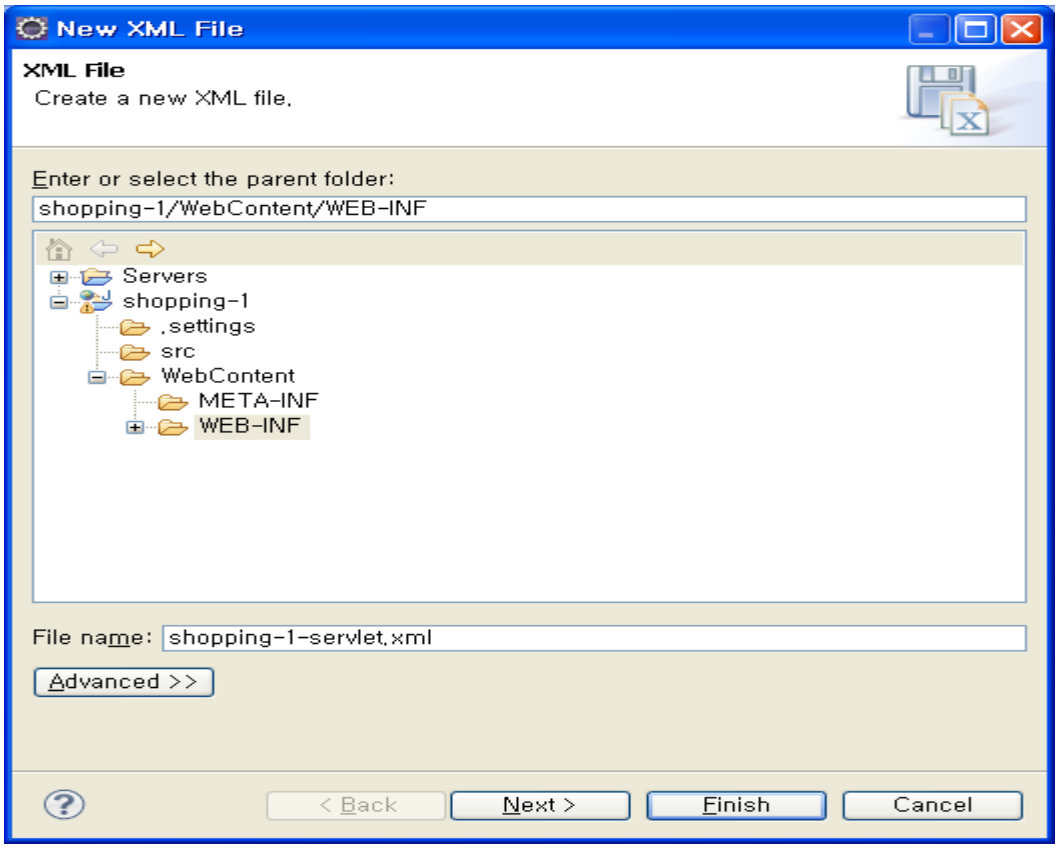
한글처리 필터적용

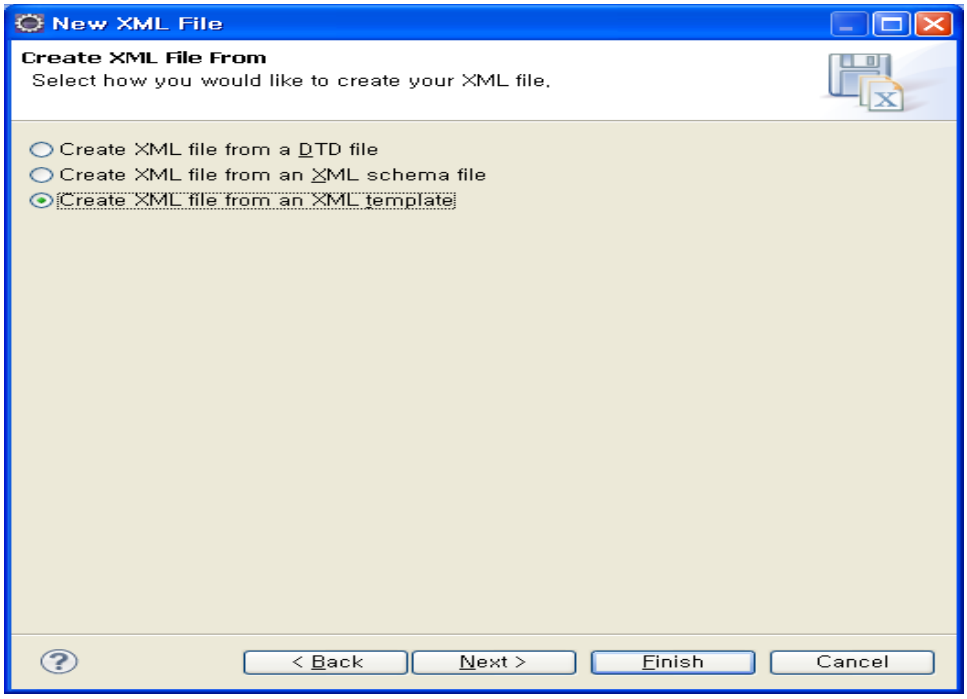
```
<filter>
    <filter-name>characterEncodingFilter</filter-name>
    <filter-class>org.springframework.web.filter.CharacterEncodingFilter
    </filter-class>
    <init-param>
        <param-name>encoding</param-name>
        <param-value>EUC-KR</param-value>
    </init-param>
    <init-param>
        <param-name>forceEncoding</param-name>
        <param-value>true</param-value>
    </init-param>
</filter>

<filter-mapping>
    <filter-name>CharacterEncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

뒤에다가 추가 =>요청이 들어왔을 때 처리해주는 DispatcherServlet과 요청 명령어를 설정한다.

```
        <filter-name>CharacterEncodingFilter</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

  <servlet>
    <servlet-name>shopping-1</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
    <servlet-name>shopping-1</servlet-name>
    <url-pattern>*.do</url-pattern>
  </servlet-mapping>
</web-app>
```

Spring에서 shopping-1에 -servelt.xml라는 문자열을 부가해서 환경설정파일이 결정이 남=>shopping-1-servelt.xml파일을 생성





Shopping-1-servlet.xml 설정파일

=>스프링 MVC용 설정파일

=>스프링의 DI컨테이너용의 XML파일과 동일한 기술을 따른다.

=>스프링 프레임워크가 이 파일을 읽어들여 MVC를 처리한다.

Index.do라고 요청이 들어오면 IndexController으로 흐름제어

```
<?xml version="1.0" encoding="EUC-KR"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
<!-- HandlerMapping -->
  <!-- 설정없음 -->

  <!-- Controller -->
  <bean id="indexController" name="/index.do"
  class="controller.IndexController">
    <property name="shopService">
      <ref bean="shopService" />
    </property>
  </bean>

  <!-- ViewResolver -->
  <!-- 설정없음 -->
```

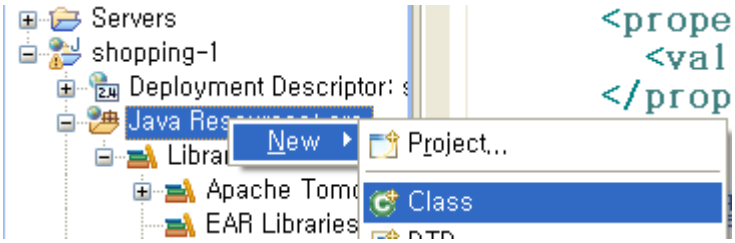
DB에 대한 환경설정부분 처리(Spring 데이터베이스,계정,암호설정)

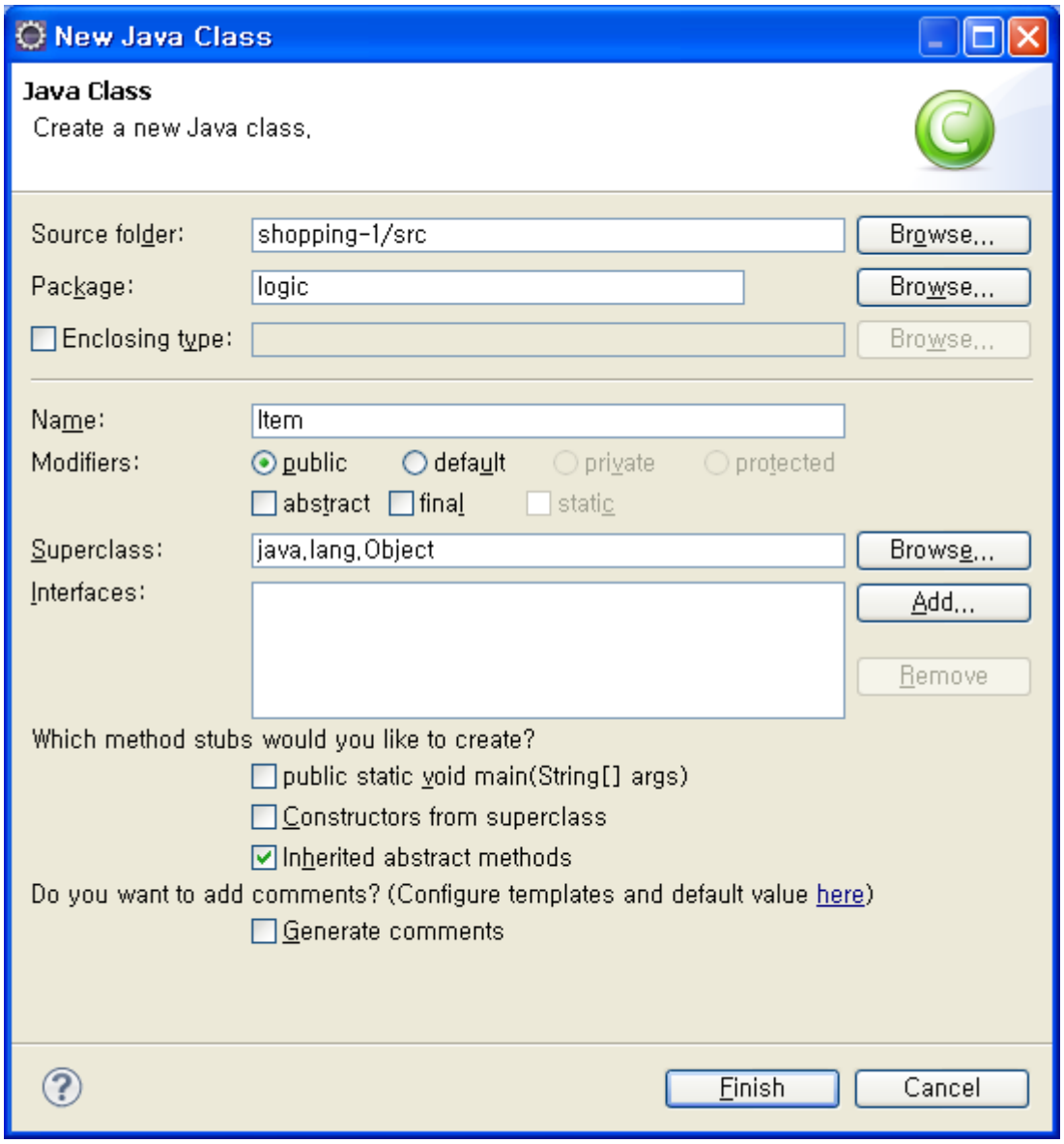
```
<!-- ViewResolver -->
<!-- 설정없음 -->

<!-- Data Source -->
<bean id="dataSource"
      class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName">
          <value>com.mysql.jdbc.Driver</value>
      </property>
  <property name="url">
    <value>jdbc:mysql://localhost:3306/spring</value>
  </property>
  <property name="username">
    <value>root</value>
  </property>
  <property name="password">
    <value>1234</value>
  </property>
</bean>
```

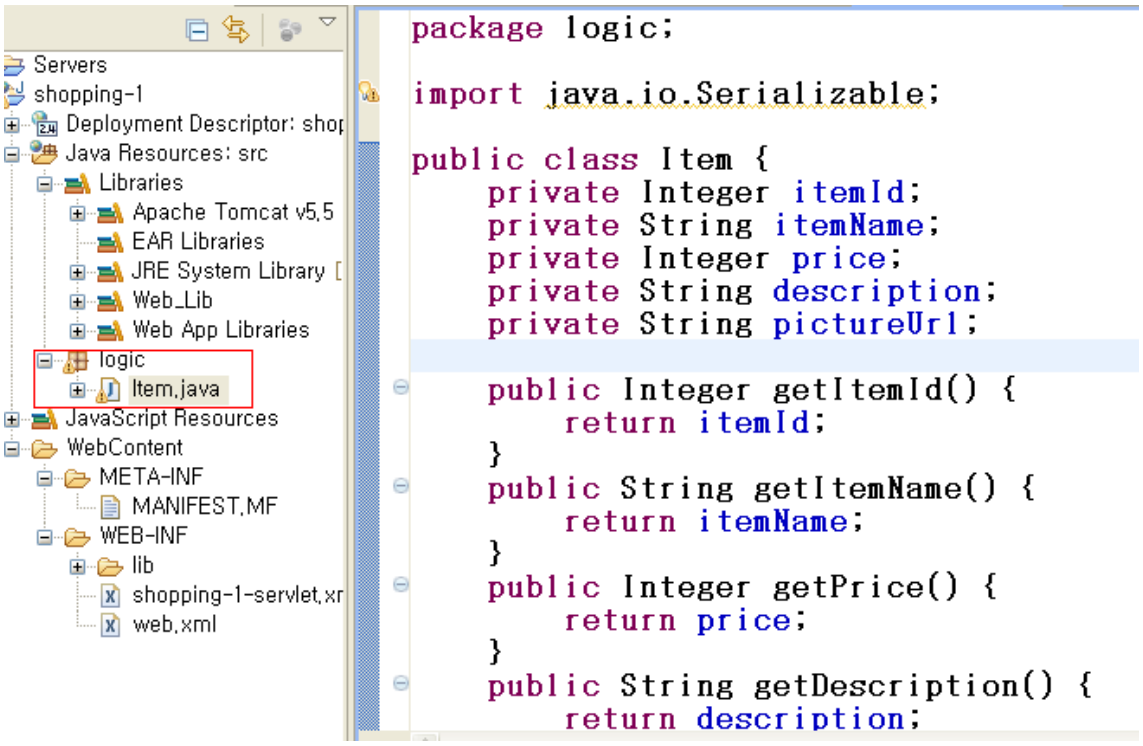
DB설정 뒤에 빈즈에 대한 3개의 설정부분(Shop->ItemCatalog->ItemDao를 참조)

```
<!-- Shop -->
<bean id="shopService" class="logic.ShopImpl">
  <property name="itemCatalog">
    <ref bean="itemCatalog" />
  </property>
</bean>

<!-- ItemCatalog -->
<bean id="itemCatalog" class="logic.ItemCatalogImpl">
  <property name="itemDao">
    <ref bean="itemDao" />
  </property>
</bean>

<!-- ItemDao -->
<bean id="itemDao" class="dao.ItemDaoImpl">
  <property name="dataSource">
    <ref bean="dataSource" />
  </property>
</bean>
</beans>
```

DAO모델에 해당하는 클래스 먼저 작성







한글화 처리 클래스 작성=>하지 않아도 된다.

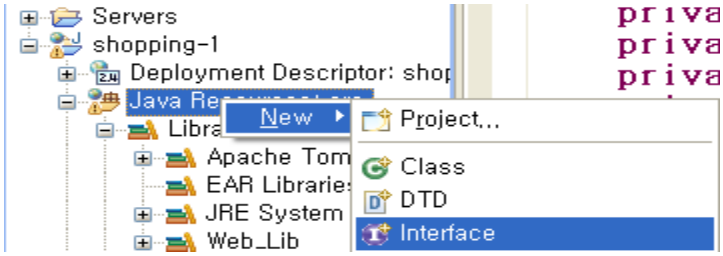
Item.java파일의 getItemName()에서 한글처리를 하지 않는다.=>한글처리하면 한글이 깨짐

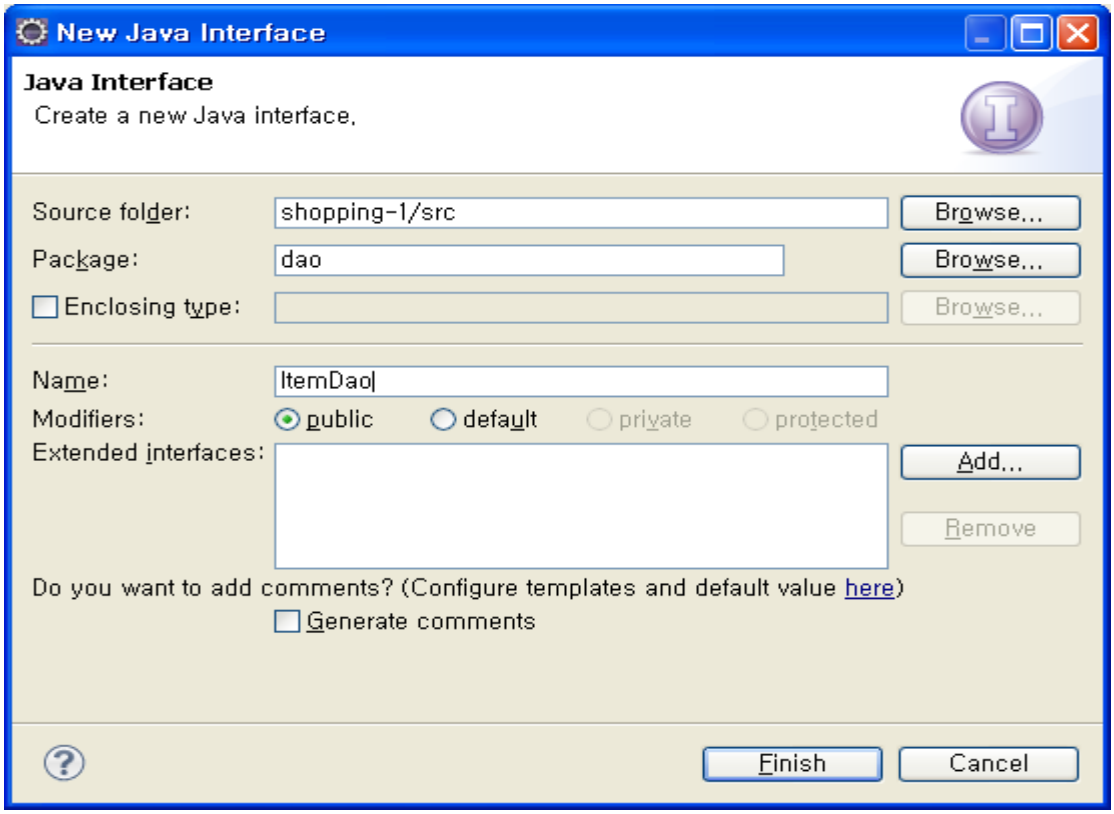
```
    }
    public String getItemName() {
        //한글처리 메서드호출
        System.out.println("저장된값을 출력(itemName)="+itemName);
        //return UiUtils.toUnicode(itemName);
        return itemName;
    }
```

setItemName()와 setDescription(),getDescription()에 한글처리를 하지 않아야 된다.

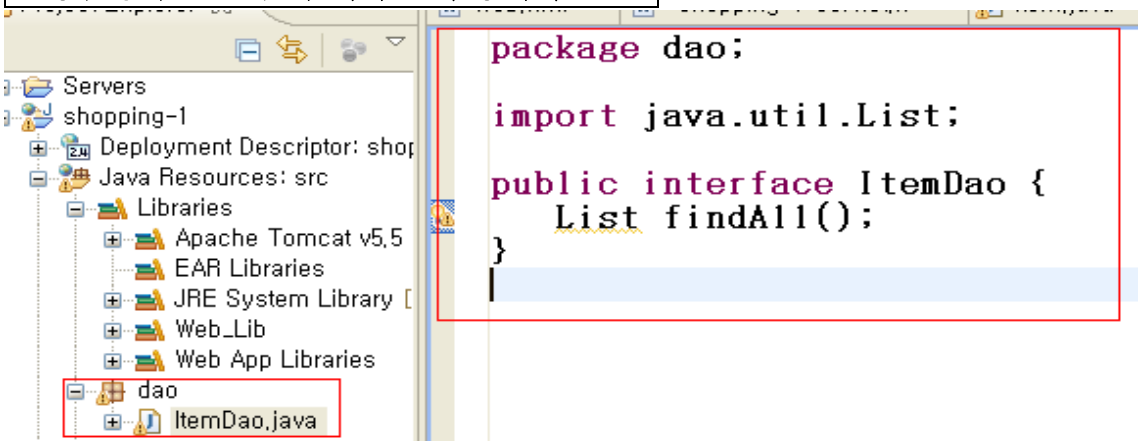
```
    public void setItemName(String itemName) {
        //System.out.println("한글처리 적용전itemName="+itemName);
        //this.itemName = UiUtils.toUnicode(itemName);
        this.itemName = itemName;
        //System.out.println("한글처리 적용후itemName="+itemName);
    }
    public void setPrice(Integer price) {
        this.price = price;
    }
    public void setDescription(String description) {
        //System.out.println("한글처리 적용전description="+description);
        //this.description = UiUtils.toUnicode(description);
        this.description = description;
        //System.out.println("한글처리 적용후description="+description);
    }
    public void setPictureUrl(String pictureUrl) {
```

상품DAO 인터페이스

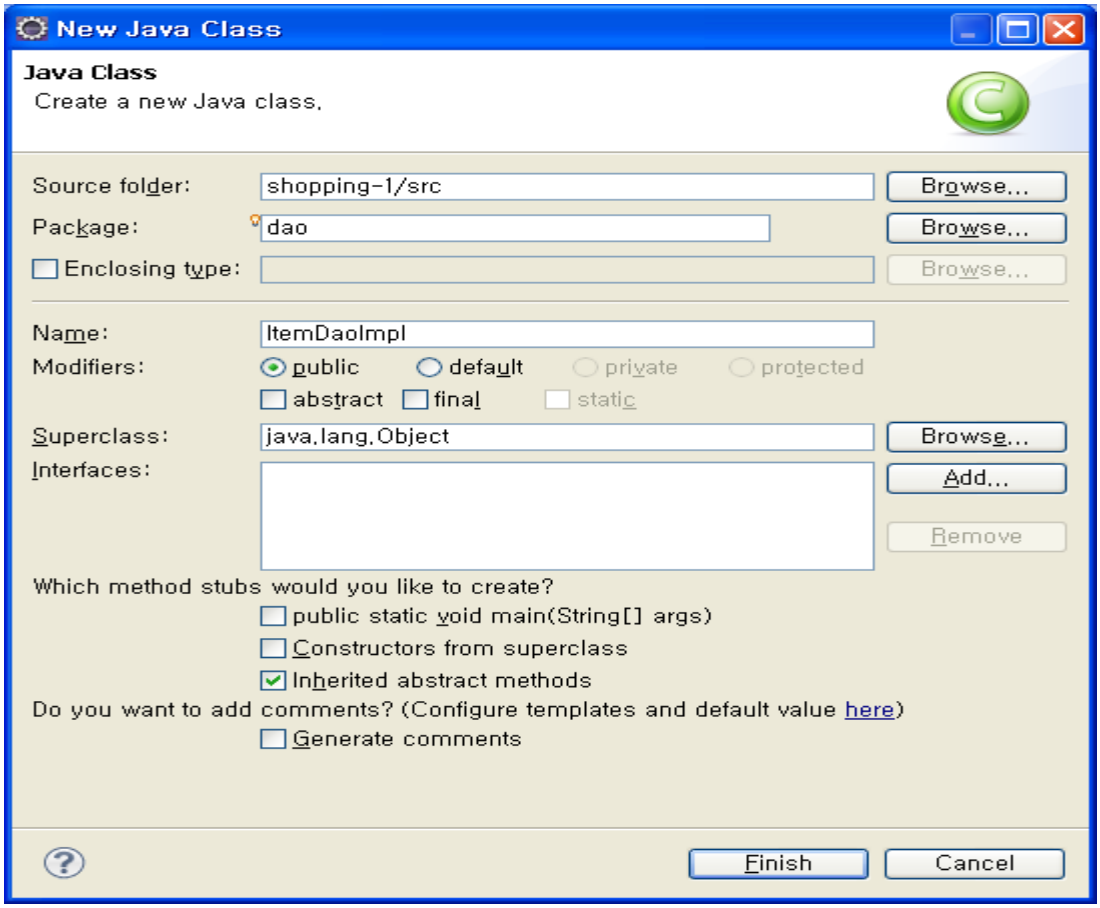


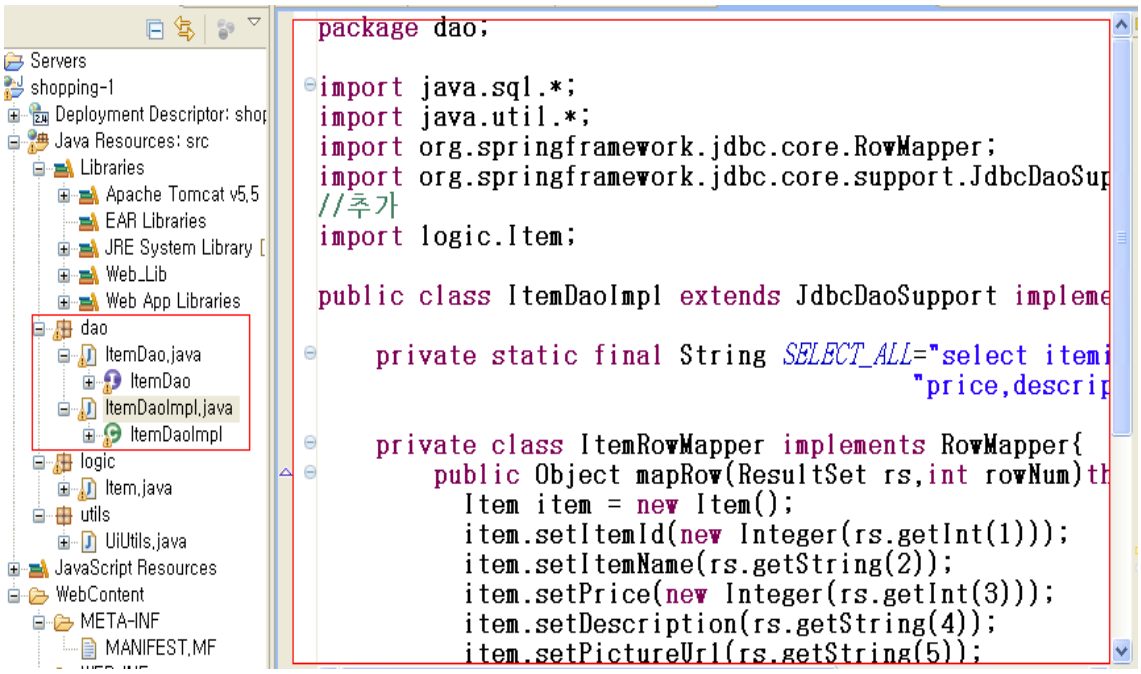


=>상품항목을 모두 검색하는 추상메서드선언

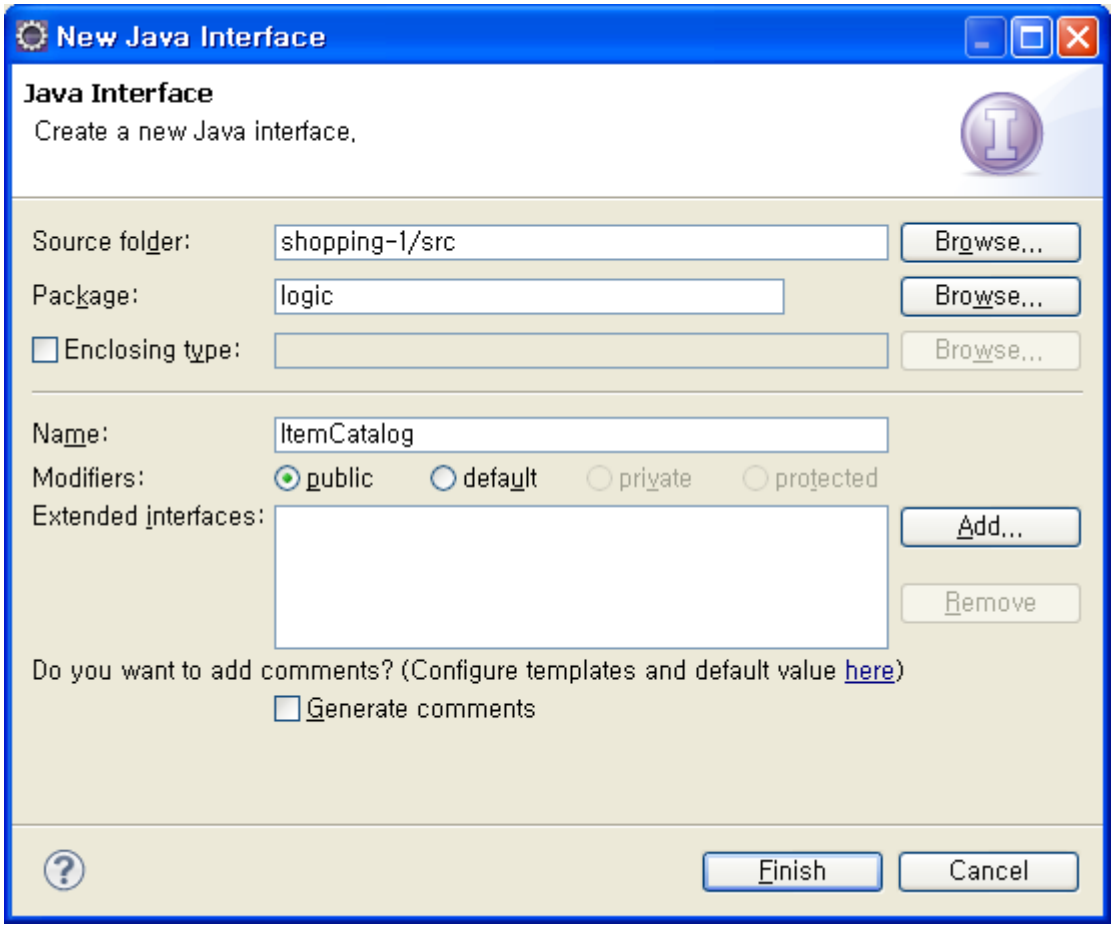


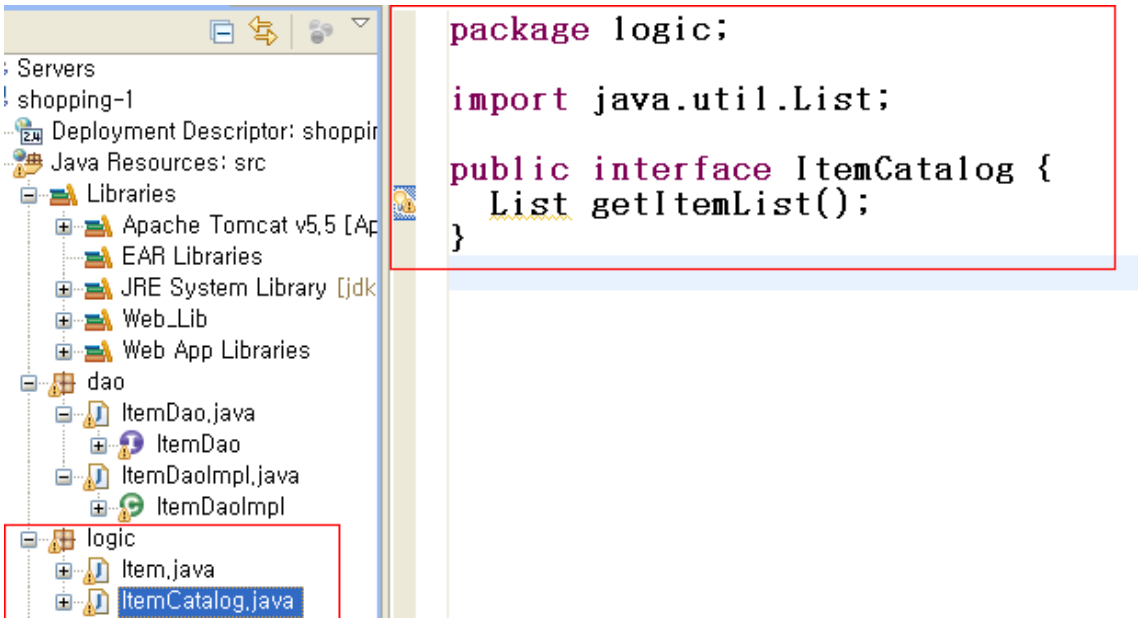
ItemDAO 인터페이스(ItemDaoImpl)구현



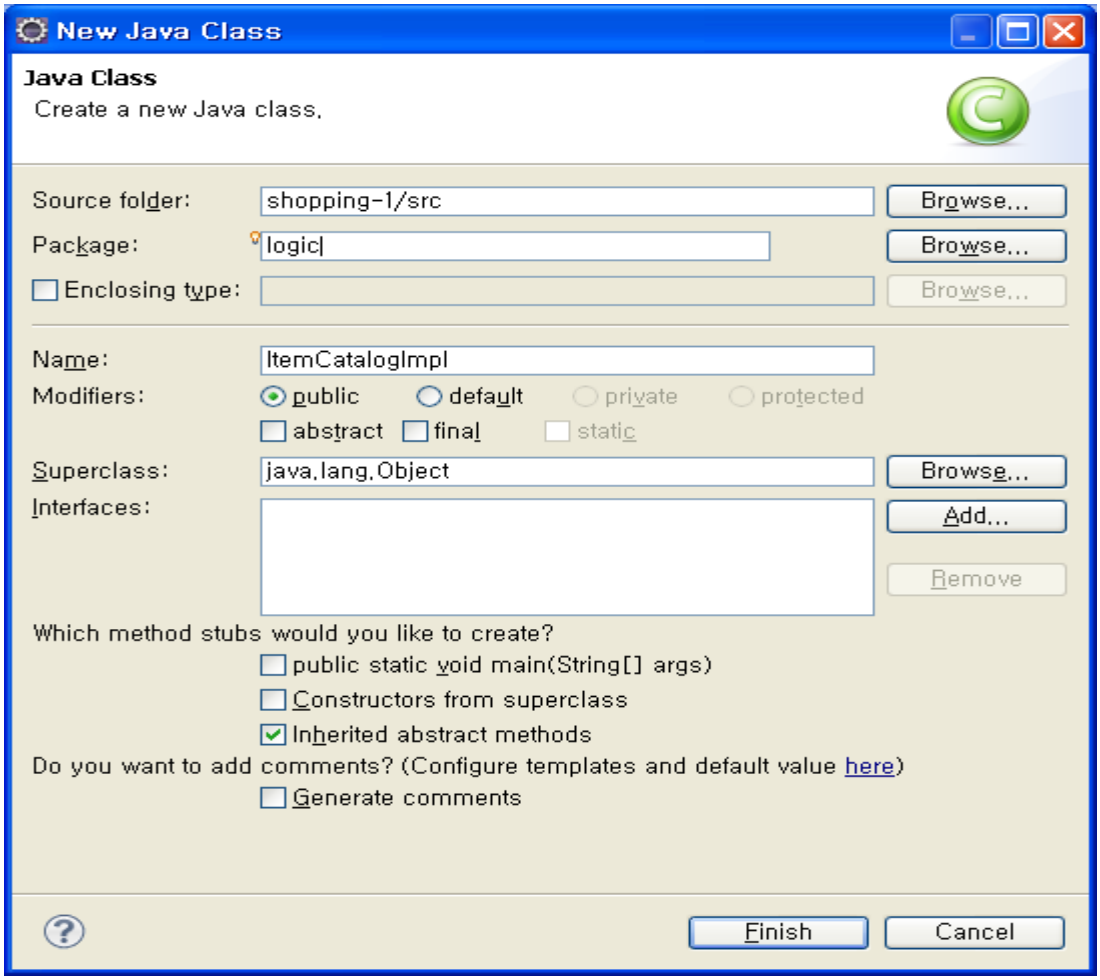


ItemCatalog인터페이스 작성



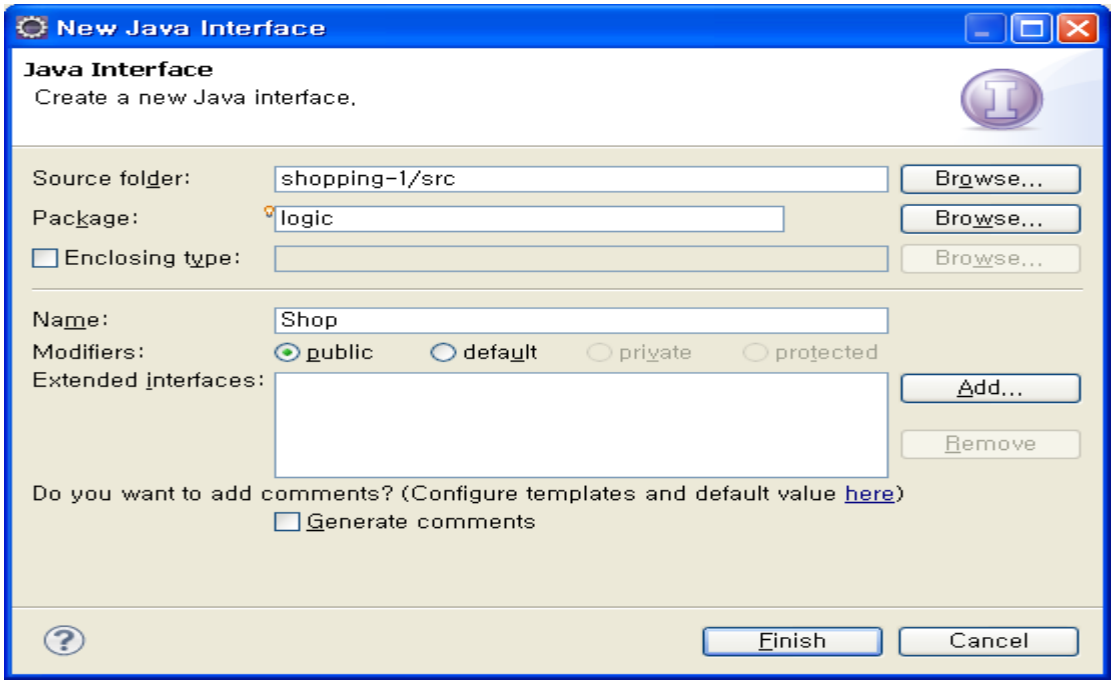


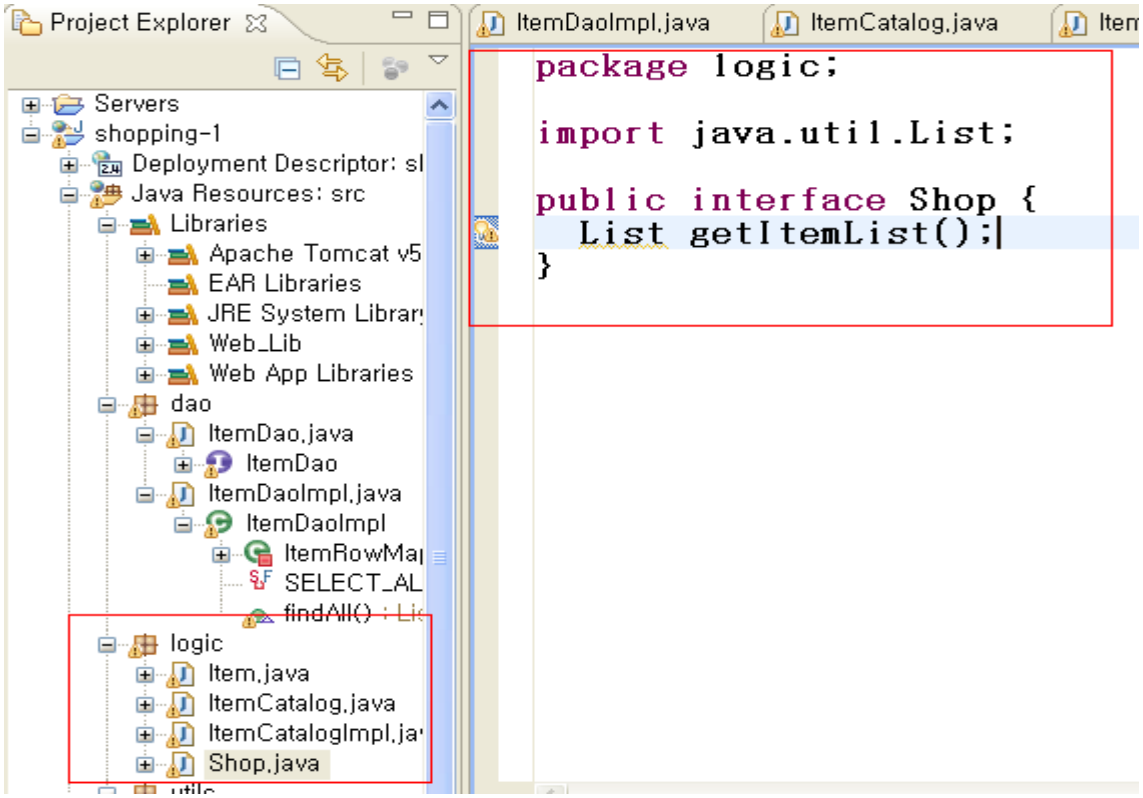
상품 카탈로그 인터페이스 구현(ItemCatalogImpl)클래스



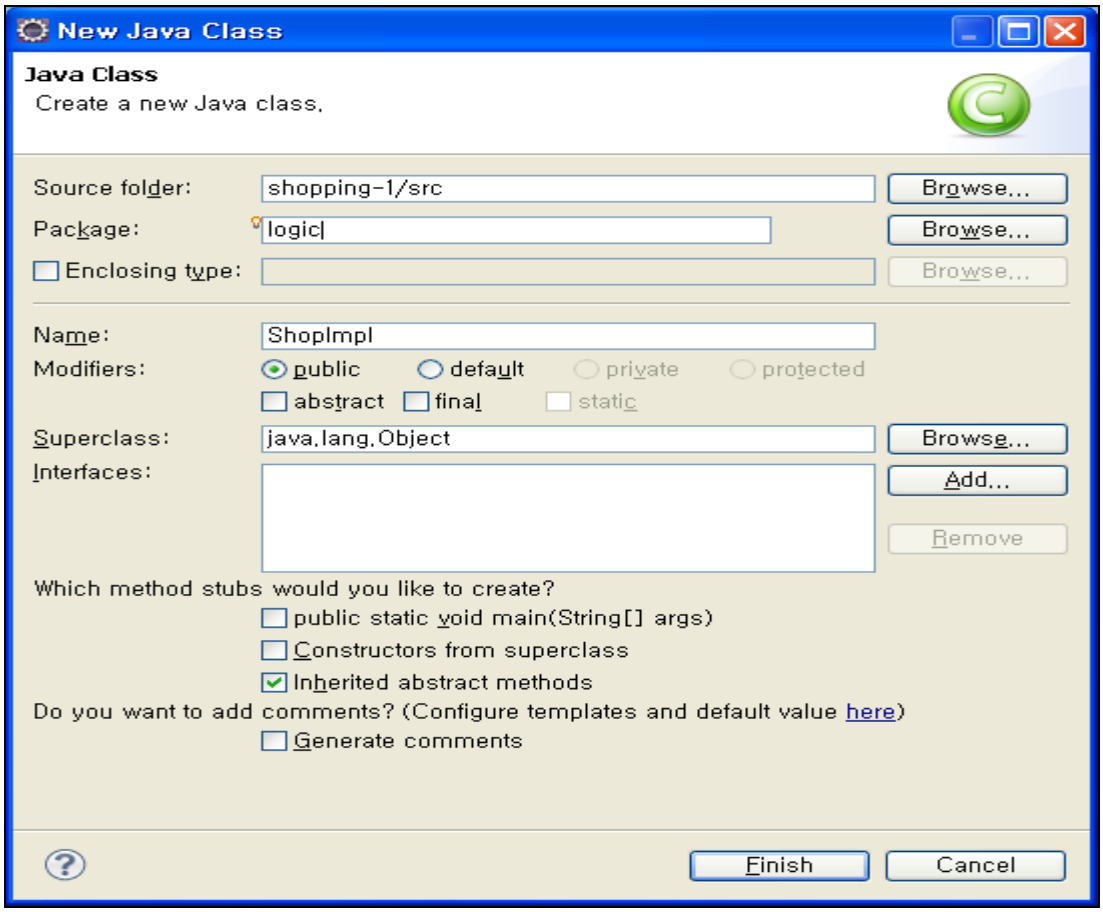

```
package logic;

import java.util.List;
import dao.ItemDao;

public class ItemCatalogImpl implements ItemCatalog{

    private ItemDao itemDao;
    public void setItemDao(ItemDao itemDao){
        this.itemDao = itemDao;
    }

    @Override
    public List getItemList(){
        return this.itemDao.findAll();
    }
}
```

Shop인터페이스 작성



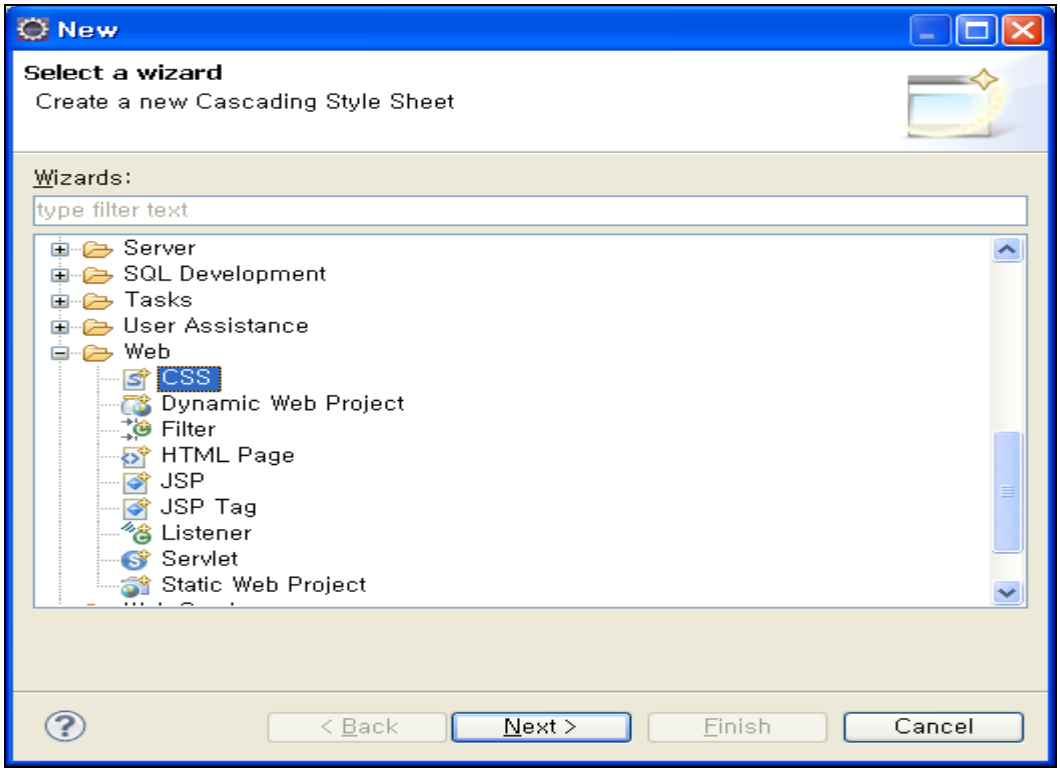


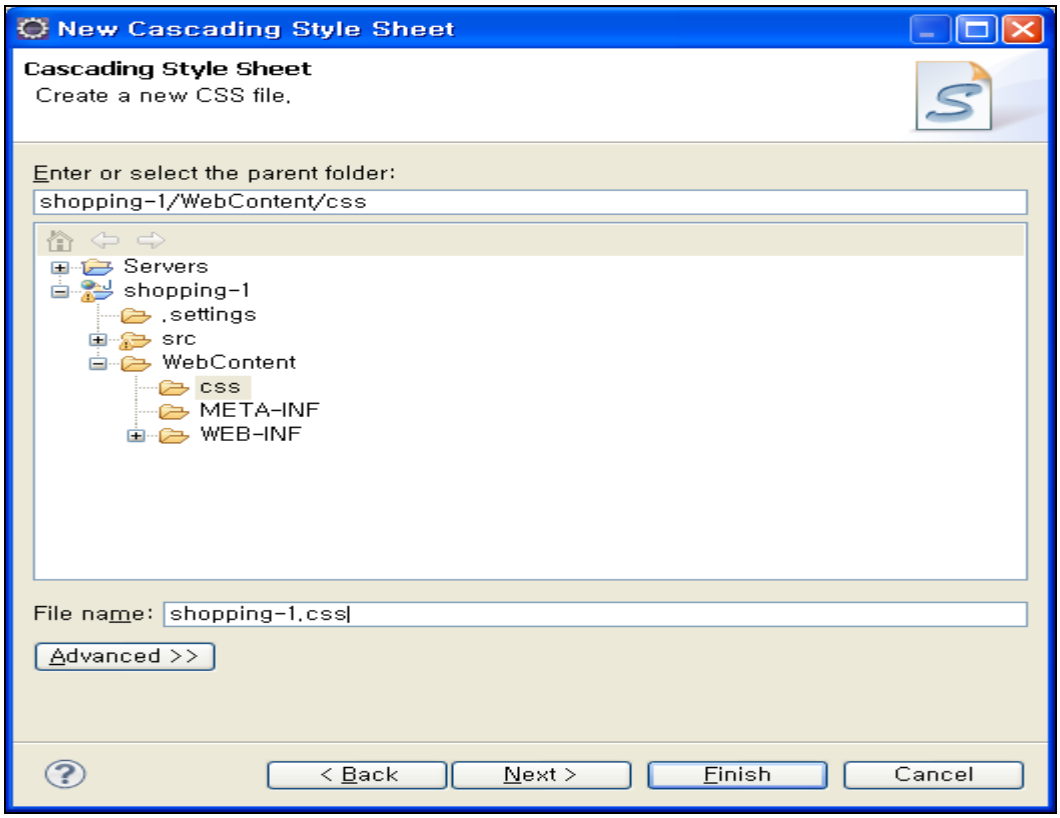
Shop인터페이스를 구현 클래스(ShopImpl)

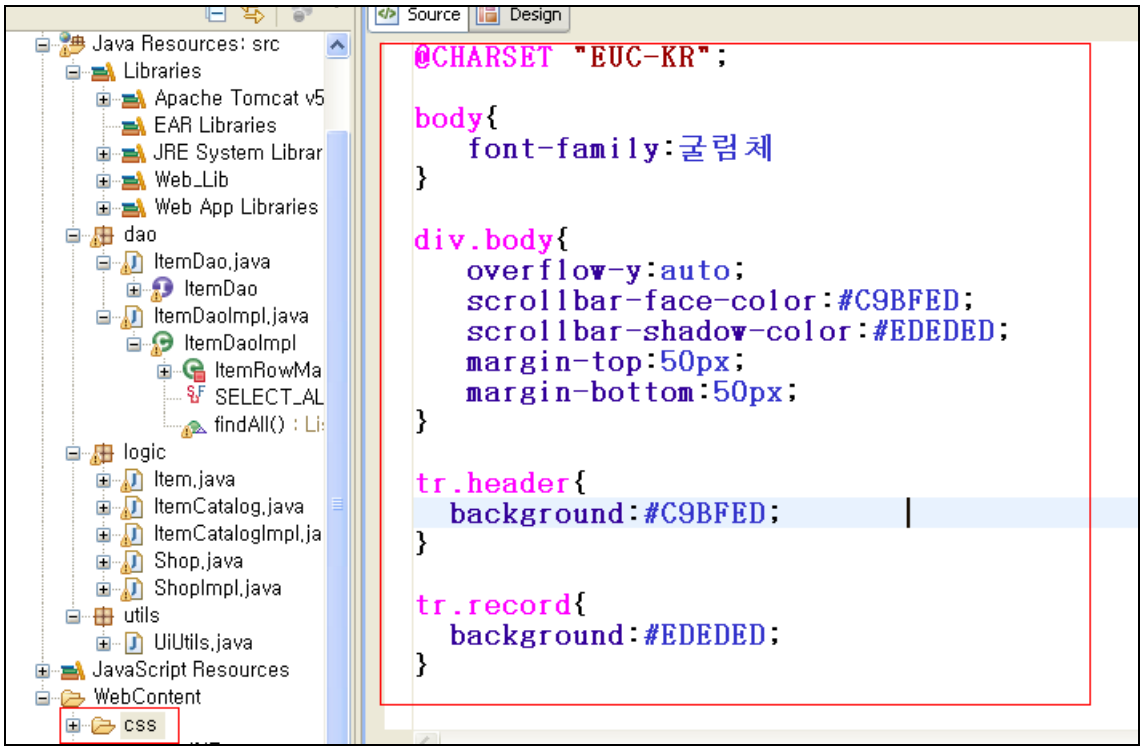


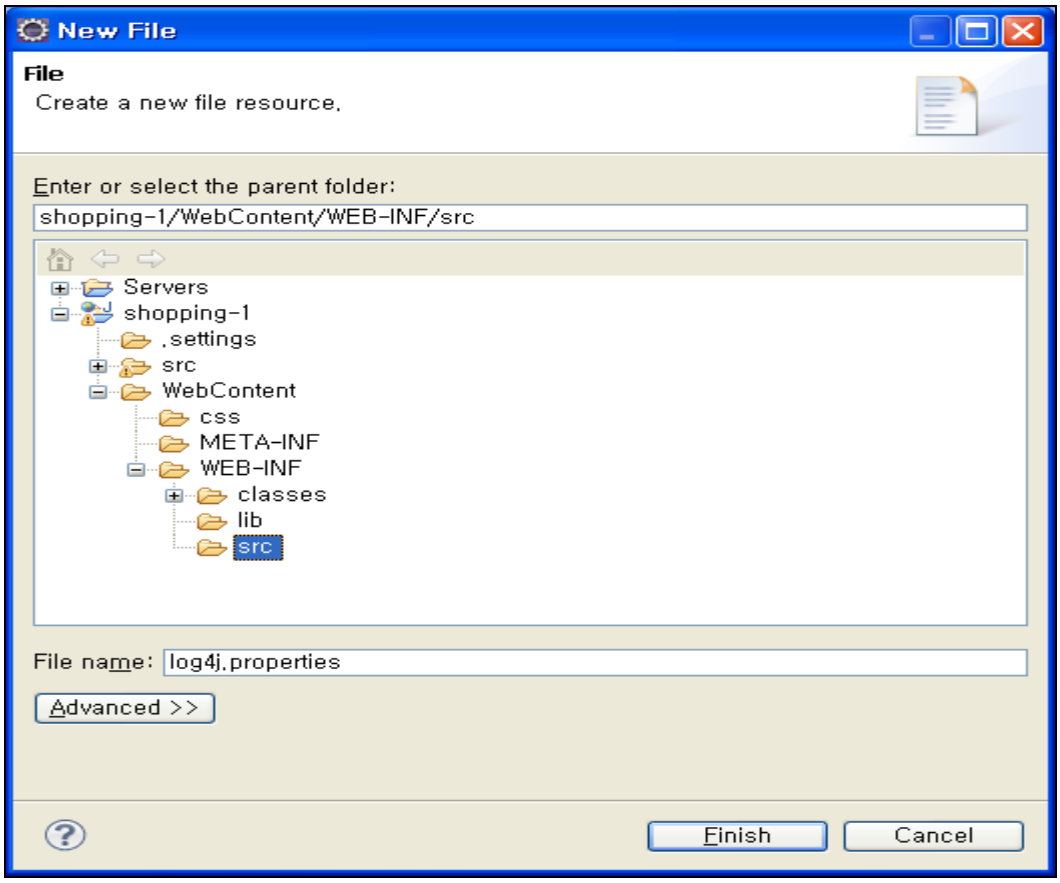

```java
package logic;

import java.util.List;

public class ShopImpl implements Shop{

    private ItemCatalog itemCatalog;
    public void setItemCatalog(ItemCatalog itemCatalog)
        this.itemCatalog = itemCatalog;
    }

    @Override
    public List getItemList(){
        return this.itemCatalog.getItemList();
    }
}
```

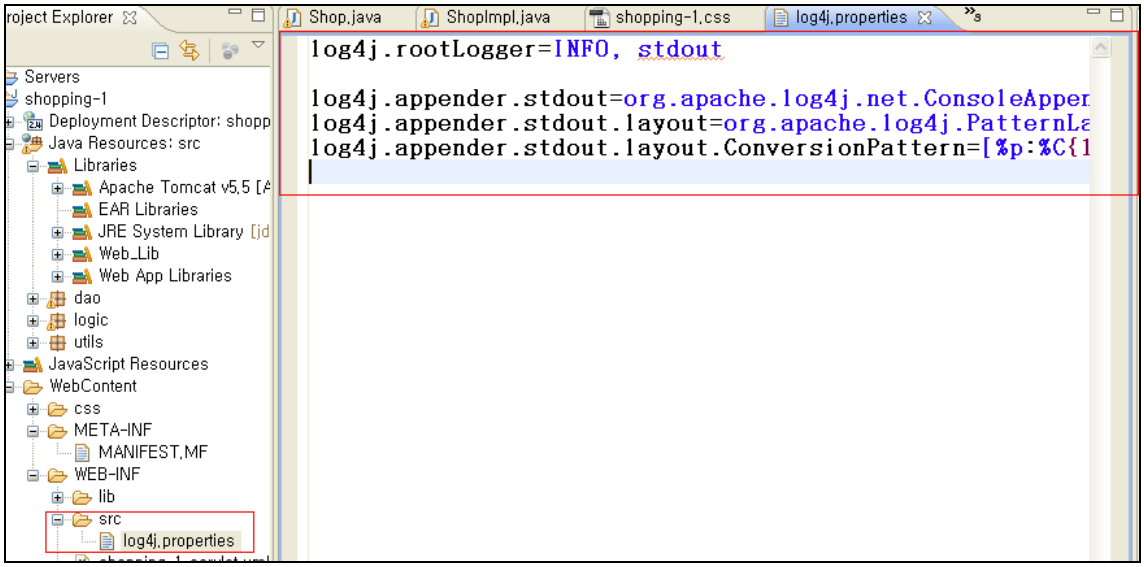
스타일시트(Shopping-1.css)

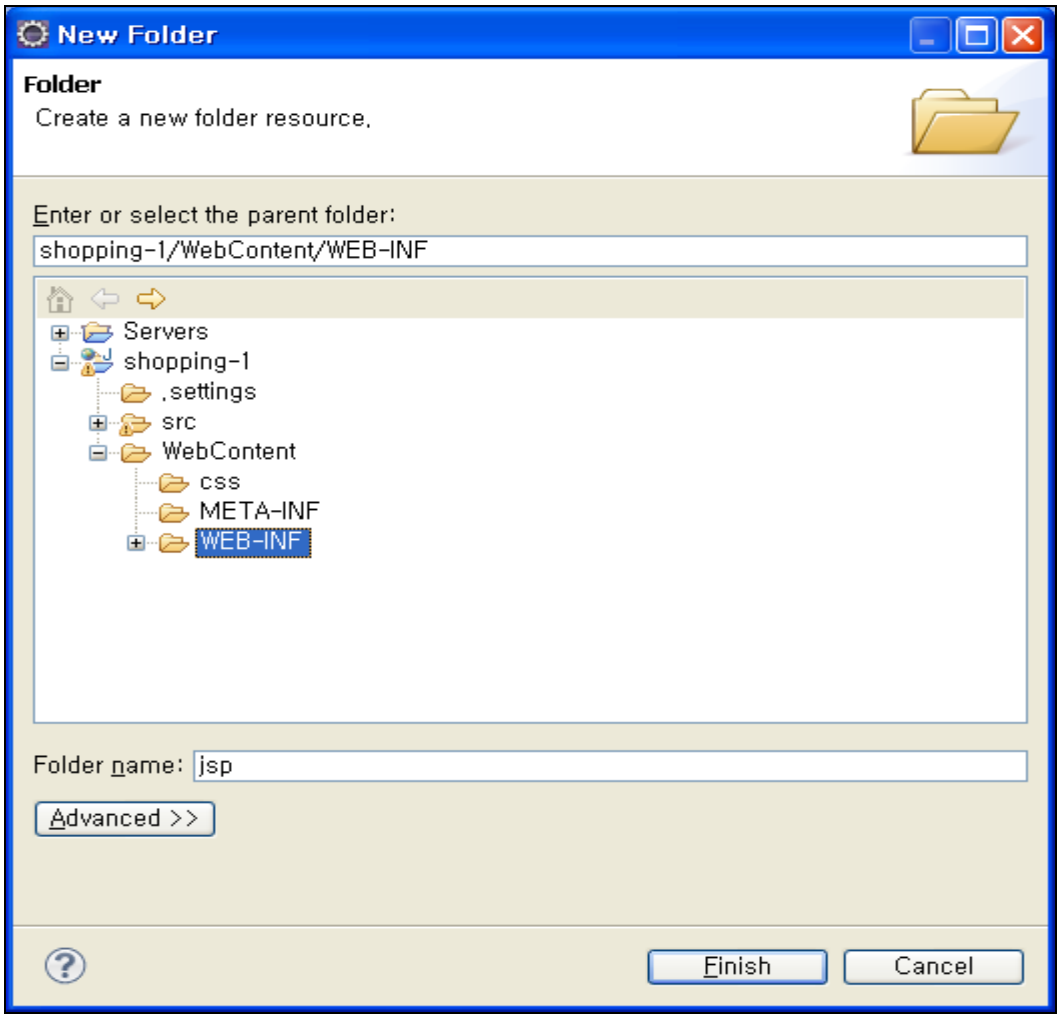


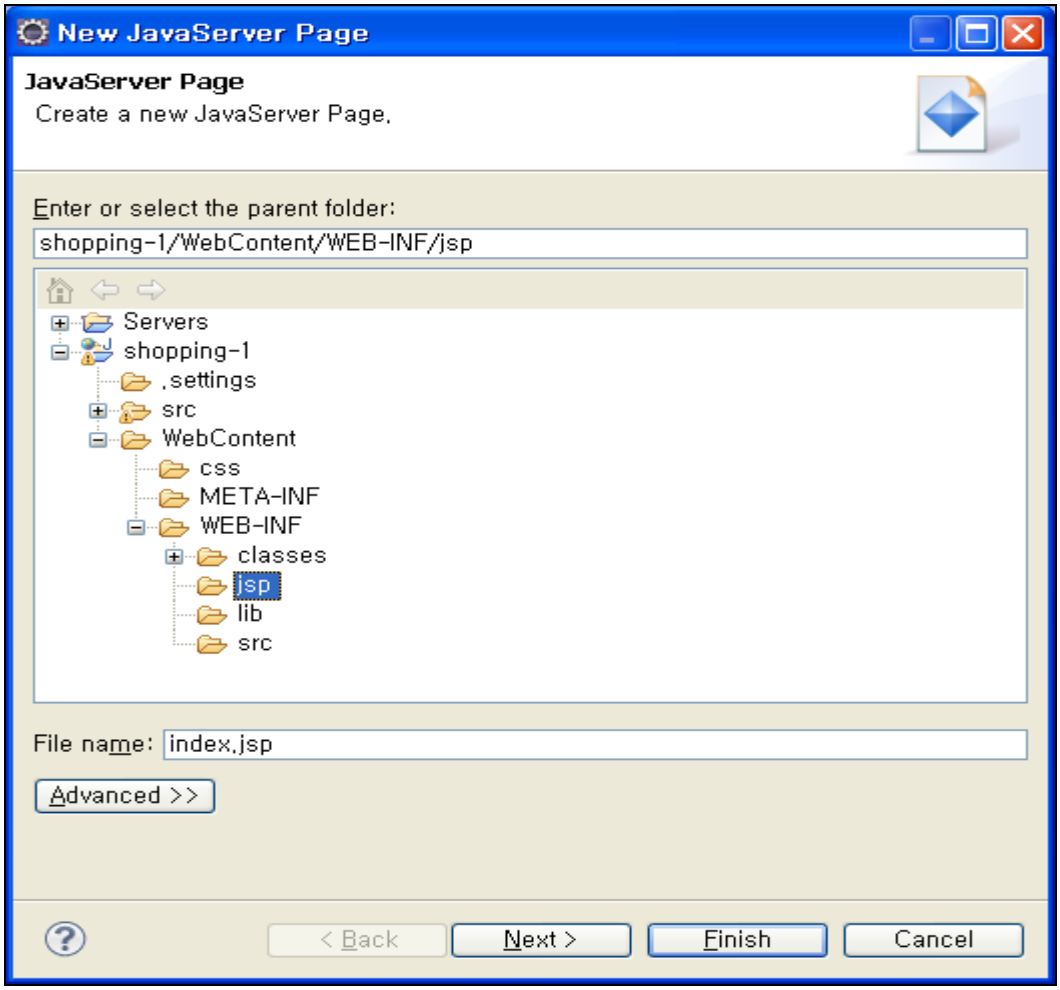



Source
Design
Java Resources: src
Libraries
Apache Tomcat v5
EAR Libraries
JRE System Librar
Web_Lib
Web App Libraries
dao
ItemDao.java
ItemDao
ItemDaoImpl.java
ItemDaoImpl
ItemRowMa
SELECT_AL
findAll() : Li
logic
Item.java
ItemCatalog.java
ItemCatalogImpl.ja
Shop.java
ShopImpl.java
utils
UiUtils.java
JavaScript Resources
WebContent
css
@CHARSET "EUC-KR";
body{
font-family:굴림체
}
div.body{
overflow-y:auto;
scrollbar-face-color:#C9BFED;
scrollbar-shadow-color:#EDEDED;
margin-top:50px;
margin-bottom:50px;
}
tr.header{
background:#C9BFED;
}
tr.record{
background:#EDEDED;
}


New File
File
Create a new file resource.
Enter or select the parent folder:
shopping-1/WebContent/WEB-INF/src
Servers
shopping-1
.settings
src
WebContent
css
META-INF
WEB-INF
classes
lib
src
File name: log4j.properties
Advanced >>
Finish
Cancel


roject Explorer
Servers
shopping-1
Deployment Descriptor: shopp
Java Resources: src
Libraries
Apache Tomcat v5.5 [A
EAR Libraries
JRE System Library [jd
Web_Lib
Web App Libraries
dao
logic
utils
JavaScript Resources
WebContent
css
META-INF
MANIFEST.MF
WEB-INF
lib
src
log4j.properties
Shop.java
ShopImpl.java
shopping-1.css
log4j.properties
log4j.rootLogger=INFO, stdout
log4j.appender.stdout=org.apache.log4j.net.ConsoleAppen
log4j.appender.stdout.layout=org.apache.log4j.PatternLa
log4j.appender.stdout.layout.ConversionPattern=[%p:%C{1

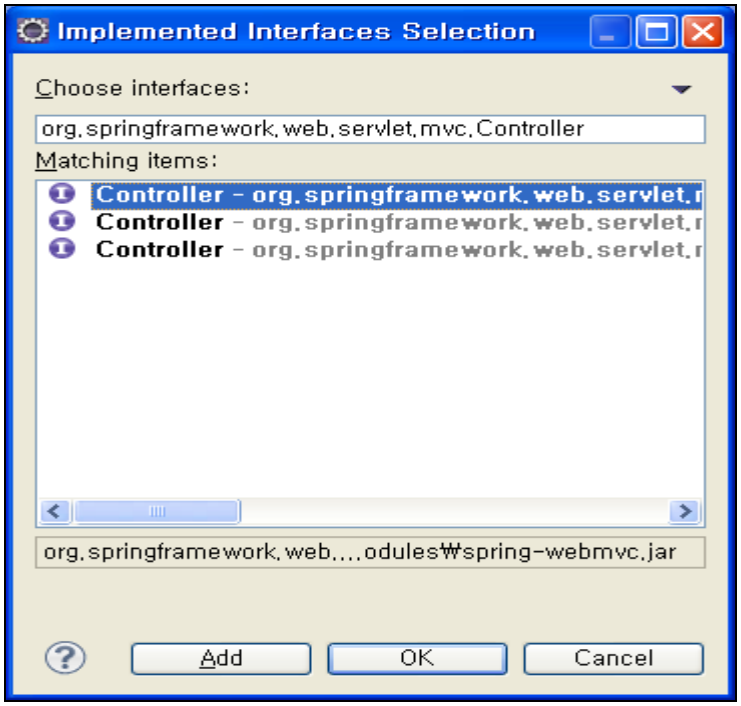

New Folder
Folder
Create a new folder resource.
Enter or select the parent folder:
shopping-1/WebContent/WEB-INF
Servers
shopping-1
.settings
src
WebContent
css
META-INF
WEB-INF
Folder name: jsp
Advanced >>
Finish
Cancel

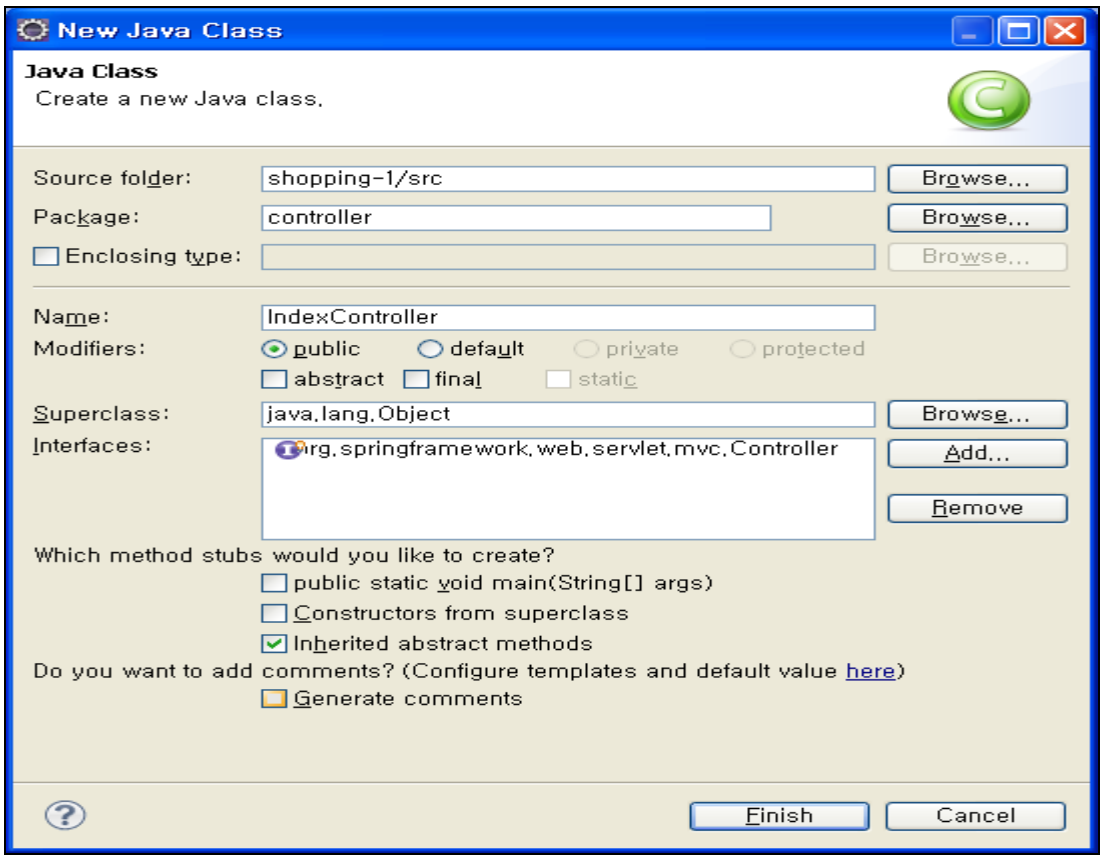


Index.jsp로 el문법이 제대로 되기위해서 설정

```
<%@ page contentType="text/html;charset=EUC-KR" %>
<%@ page session="false" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ page import="java.util.*" %>
<%@ page isELIgnored="false"%>

<link rel="stylesheet" type="text/css" href="css/shopping-1.css">
<html>
<head>
    <title>상품 리스트 화면</title>
</head>
<body>
    <div align="center" class="body">
    <h2>상품 리스트 화면</h2>
    <table border="1">
        <tr class="header">
            <th align="center" width="100">상품 ID</th>
            <th align="center" width="320">상품 명</th>
            <th align="center" width="100">가격</th>
        </tr>
```

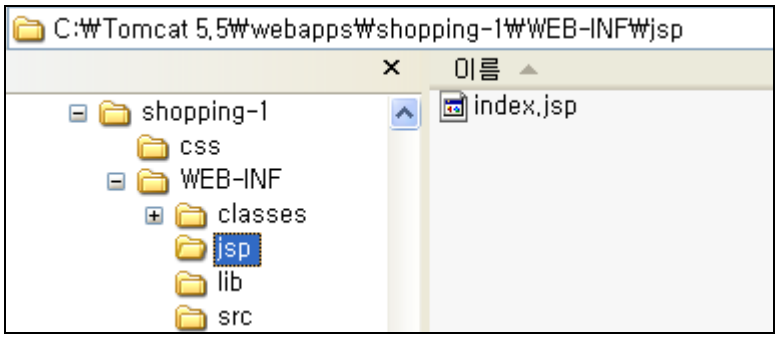
Controller작성





```java
import org.springframework.web.servlet.mvc.Controller;
//추가
import logic.Shop;

public class IndexController implements Controller {

    private Shop shopService;
    public void setShopService(Shop shopService){
        this.shopService = shopService;
    }

    @Override
    public ModelAndView handleRequest(HttpServletRequest request,
            HttpServletResponse response) throws Exception {
        // TODO Auto-generated method stub
        //상품 리스트 정보 취득
        List itemList = this.shopService.getItemList();

        //모델작성
        Map model = new HashMap();
        model.put("itemList", itemList);

        //반환값이 되는 ModelAndView인스턴스 작성
        ModelAndView modelAndView = new ModelAndView();
        modelAndView.setViewName("/WEB-INF/jsp/index.jsp");
        modelAndView.addAllObjects(model);
        return modelAndView;
    }
}
```

이클립스에서 에러가 발생하여 그냥 톰캣서버에서 실행시킴.



<한글 문제 해결>

오히려 한글처리를 하지 않아야 한글이 깨지지 않음을 확인했음

```
public void setItemName(String itemName) {
    //System.out.println("한글처리 적용전itemName="+itemName);
    //this.itemName = UiUtils.toUnicode(itemName);
    this.itemName = itemName;
    //System.out.println("한글처리 적용후itemName="+itemName);
}
public void setPrice(Integer price) {
    this.price = price;
}
public void setDescription(String description) {
    //System.out.println("한글처리 적용전description="+description)
    //this.description = UiUtils.toUnicode(description);
    this.description = description;
    //System.out.println("한글처리 적용후description="+description)
}
```

```
}
public String getItemName() {
    //한글처리 메서드호출
    System.out.println("저장된값을 출력(itemName)="+itemName)
    //return UiUtils.toUnicode(itemName);
    return itemName;
}
```

<최종실습>

http://localhost:8089/shopping-1/index.do

상품 리스트 화면

상품 ID	상품 명	가격
2	오렌지	2000원
3	키위	300원
4	파란사과	500원
5	블루베리	500원
6	체리	1000원
7	메론	1000원
8	수박	2000원
9	파인애플	2000원